Classical ML & Decision Science

1. Why this module matters

A large share of enterprise value (churn, demand forecasting, fraud, recommendations, risk scoring) is not an LLM problem. The most senior move in many engagements is talking the client **out of** an LLM.

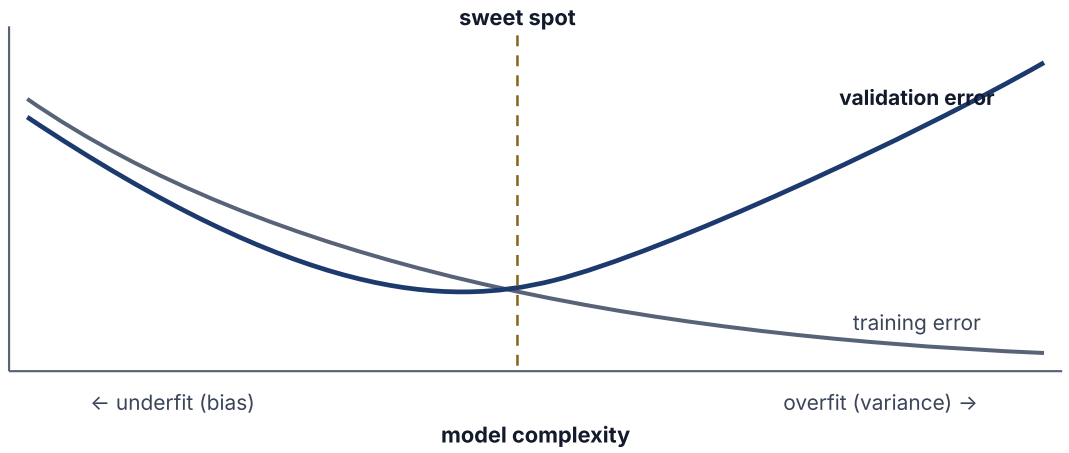
2. Vocabulary

- **Features** = the input variables · **Label / target** = what you predict
- **Training** = fitting the model to labelled history · **Inference / scoring** = applying it to new cases
- **Supervised** = you have labelled outcomes · **Unsupervised** = no labels, you look for structure (e.g. customer segments)
- **Classification** predicts a category (churn or not); **regression** predicts a number (next month's revenue)
- Most classifiers output a probability; the **threshold** that turns it into a yes/no is a business decision about the cost of false alarms vs missed cases

3. Overfitting and honest testing

- Too simple → **underfits** (high bias). Too complex → memorises noise and **overfits** (high variance): perfect on training data, poor on new data
- Spot overfitting as a big gap between training and validation scores
- Split data into **train / validate / test**, keep the test set untouched until the end, and use cross-validation to tune

✎ DIAGRAM



Training error keeps falling as the model gets more complex; validation error falls then rises. The sweet spot is the bottom of the validation curve.

4. Metrics that matter

- **Accuracy trap**: a fraud model that always says 'legitimate' scores 99.9% and catches nothing
- **Precision**: of everything I flagged, how much was really positive? (are the alarms trustworthy?)
- **Recall**: of all the real positives, how many did I catch?
- **F1** balances the two · **AUC** measures separation across all thresholds · **MAE / RMSE** for regression (RMSE punishes big misses more)
- The consultant's job: translate precision vs recall into money (cost of a false alarm vs cost of a missed case)

✎ DIAGRAM

	Model says YES	Model says NO
Really YES	True positive	False negative (missed)
Really NO	False positive (false alarm)	True negative

Precision = TP ÷ (TP + FP). Recall = TP ÷ (TP + FN).

5. Model families

- **Logistic / linear regression**: the interpretable baseline you never skip
- **Gradient-boosted trees** (XGBoost, LightGBM, CatBoost): trees built in sequence, each fixing the last one's errors. **Still state of the art on tabular business data**, fast and explainable

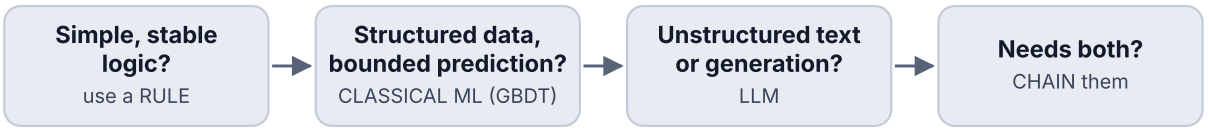
- **Random forest**: robust, low-tuning default · **k-means**: unsupervised segmentation
- **Neural nets**: images, audio, text. Usually the wrong, expensive choice for a spreadsheet

6. Causation, A/B tests and uplift

- A predictive model finds **correlations**: enough to forecast, never enough to justify an intervention (loyal customers using feature X doesn't mean X causes loyalty)
- **A/B test**: randomly split into treatment and control; because assignment is random, any difference is caused by the change. Pitfalls: no pre-set metric, too small a sample, **peeking**
- Can't randomise? Quasi-experiments (difference-in-differences, regression discontinuity), with weaker assumptions
- **Uplift modelling**: target the **persuadables** (people the action actually changes), not the lost causes or the sure things

7. The decision: rule, ML or LLM

✎ DIAGRAM



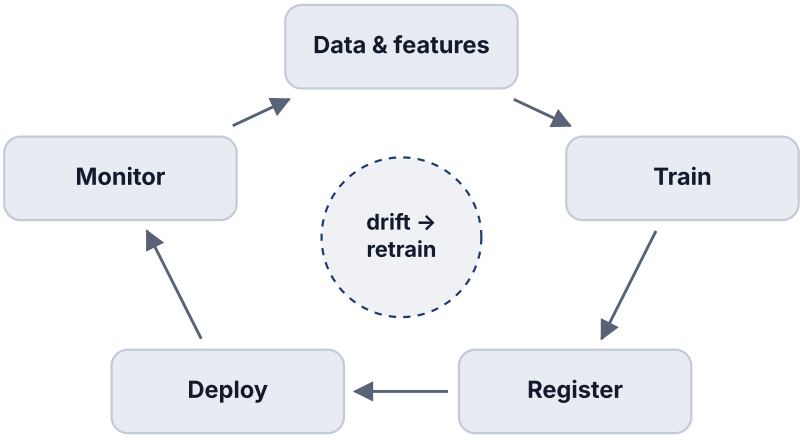
Ask in order, and always ask: would something cheaper be nearly as good?

8. MLOps for classical models

1. Features produced by a scheduled pipeline
2. Train → validate → **register** the model with its data version and metrics
3. Deploy as **batch** scoring (nightly scores to a table, e.g. churn) or **real-time** scoring (an API, e.g. fraud)
4. Monitor → retrain on a schedule or when drift crosses a threshold

- **Feature store**: features defined once and served the same way for training and live scoring, preventing **training-serving skew**
- **Model registry** and **experiment tracking** (MLflow is the common name)
- **Data drift** = inputs change · **Concept drift** = the relationship changes · **Label delay** = true outcomes arrive weeks later, so monitor inputs and prediction distributions meanwhile · **Feedback loops** = the model's own actions change future data

✎ DIAGRAM



The classical ML lifecycle: monitoring closes the loop back to retraining.

SAY IT

"If the problem is a spreadsheet, my default is a boosted tree: it costs a fraction of an LLM, it's usually more accurate on tabular data, and I can explain it to a regulator."